Course Final Project: Supervised Machine Learning Classification

Dataset: Default of Credit Card Client by Monica Gayatri K.

1. Main Objective

The dataset of Default of Credit Card Client contains detailed information regarding customer default payments on credit card (CC) from a card-issuing bank in Taiwan. The primary objective of compiling this payment history is to mitigate the risk of an increasing debt crisis and the delinquency rates caused by over-issuance of credit to unqualified applicants.

The dataset comprises **30,000 records and 23 variables** detailing customer's personal information and payment histories. The creator's intent is to analyze and predict customer payment behavior based on that information and identify which classification model has the best accuracy of probability. On this project, I'll adopt this intent as the project main objectives, which entails:

1. Developing several models to predict whether a customer will default on a payment.
2. Evaluating and comparing these classification models using the following metrics: accuracy, precision, recall, F1-score, and ROC-AUC.

The classification model utilized will be strictly limiting to the following methods:

- Logistic Regression
- KNN
- Random Forest
- Gradient Boosting
- Stacking Ensemble

2. Exploration Data, Data Cleaning, and Feature Engineer

2.1 Exploration Data

2.1.1 Gathering Information

As the dataset contains 23 variables/Features, we need to get a snippet of the dataset first through data.head(). However it turned out that column 0 is showing the description of each header. To avoid error in further featuring engineer and model training, we need to replace the header with the value of column 0 for easier identification also.

	Unnamed: 0	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13
0	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5	PAY_6	BILL_AMT1	BILL_AMT2	BILL
1	1	20000	2	2	1	24	2	2	-1	-1	-2	-2	3913	3102
2	2	120000	2	2	2	26	-1	2	0	0	0	2	2682	1725
3	3	90000	2	2	2	34	0	0	0	0	0	0	29239	14027
4	4	50000	2	2	1	37	0	0	0	0	0	0	46990	48233

Fig. 1. Initial output data.head()

After changing the header name, then we can proceed to identify each type of the variable by using *data.head()* in Fig.2. to get a snippet of the dataset and *data.info()* in Fig. 3. to understand the datatype, total entry, and NaN existence.

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5	PAY_6	BILL_AMT1	BILL_AMT2	BILL_AMT3	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5	PAY_AMT6	default payment next month
0	1	20000	2	2	1	24	2	2	-1	-1	-2	-2	3913	3102	689	0	0	0	0	689	0	0	0	0	1
1	2	120000	2	2	2	26	-1	2	0	0	0	2	2682	1725	2682	3272	3455	3261	0	1000	1000	1000	0	2000	1
2	3	90000	2	2	2	34	0	0	0	0	0	0	29239	14027	13559	14331	14948	15549	1518	1500	1000	1000	1000	5000	0
3	4	50000	2	2	1	37	0	0	0	0	0	0	46990	48233	49291	28314	28959	29547	2000	2019	1200	1100	1069	1000	0
4	5	50000	1	2	1	57	-1	0	-1	0	0	0	8617	5670	35835	20940	19146	19131	2000	36681	10000	9000	689	679	0
5	6	50000	1	1	2	37	0	0	0	0	0	0	64400	57069	57608	19394	19619	20024	2500	1815	657	1000	1000	800	0

Fig.2. data.head()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   ID                          30000 non-null  object
 1   LIMIT_BAL                   30000 non-null  object
 2   SEX                         30000 non-null  object
 3   EDUCATION                   30000 non-null  object
 4   MARRIAGE                    30000 non-null  object
 5   AGE                         30000 non-null  object
 6   PAY_0                       30000 non-null  object
 7   PAY_2                       30000 non-null  object
 8   PAY_3                       30000 non-null  object
 9   PAY_4                       30000 non-null  object
 10  PAY_5                       30000 non-null  object
 11  PAY_6                       30000 non-null  object
 12  BILL_AMT1                   30000 non-null  object
 13  BILL_AMT2                   30000 non-null  object
 14  BILL_AMT3                   30000 non-null  object
 15  BILL_AMT4                   30000 non-null  object
 16  BILL_AMT5                   30000 non-null  object
 17  BILL_AMT6                   30000 non-null  object
 18  PAY_AMT1                    30000 non-null  object
 19  PAY_AMT2                    30000 non-null  object
 20  PAY_AMT3                    30000 non-null  object
 21  PAY_AMT4                    30000 non-null  object
 22  PAY_AMT5                    30000 non-null  object
 23  PAY_AMT6                    30000 non-null  object
 24  default payment next month  30000 non-null  object
dtypes: object(25)
memory usage: 5.7+ MB
```

Fig. 3. data.info()

The dataset's creator also has shared explanation for each variables, which entails:

- LIMIT_BAL: Amount of the total given credit (in dollar, $) from both the customer credit and his/her family (supplementary) credit.
- Gender: 1 = male; 2 = female.

- Education: 1 = graduate school; 2 = university; 3 = high school; 4 = others.
- Marital status: 1 = married; 2 = single; 3 = others.
- Age: Customer age in year.
- PAY_0 to PAY_6: customer payment history. (-1 = pay duly; 1 = payment delay for one month; 2 = payment delay for two months; . . .; 8 = payment delay for eight months; 9 = payment delay for nine months and above.)
- BILL_AMT1 to BILL_AMT6: amount of bill statement (in dollar, $).
- PAY_AMT1 to PAY_AMT6: amount of previous payment (in dollar, $).

2.1.2 Insight

Based on Fig. 2. And Fig. 3., the dataset has:

1. From Fig. 3. data.info(), there are total 24 columns, which entails:
    a. 23 columns of features.
    b. The 1 column of target that we need to predict is 'default payment next month'.
2. Based on the dataset's snippet Fig.2. All the values are in number, but all the data types are showing objects. We'll need to change the data type into integer in Feature engineer.
3. There is no missing value from each column.

2.2 Data Cleaning

As the checking in Fig.3., we found no missing value. In the data cleaning we'll only check if there's any duplicated values in the data set by using *len(data.duplicated())* to get the total count of non-duplicated data.

```
[44]: len(df_cc.duplicated())

[44]: 30000
```

Fig.4. len(data.duplicated())

Turns out, the dataset doesn't have any duplicated data because the output is the same as the dataset total entries.

2.3 Feature Engineer

2.3.1 Gathering Information

In the feature engineer, we'll check whether the column 'default payment next month' has balanced binary value using barchart in Fig.5. and change the datatype into int as per insight from 2.1.1 using *data.astype(int)*.

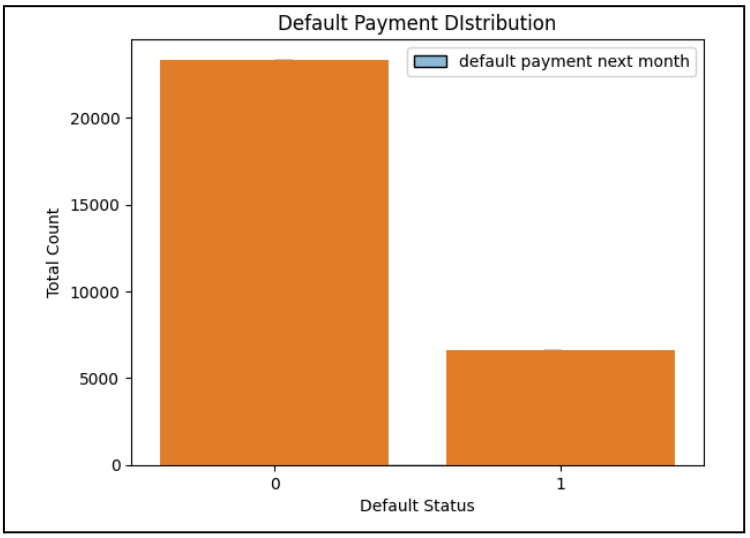

Fig.5. Bar chart of 'default payment next month'

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   ID                          30000 non-null  int64
 1   LIMIT_BAL                   30000 non-null  int64
 2   SEX                         30000 non-null  int64
 3   EDUCATION                   30000 non-null  int64
 4   MARRIAGE                    30000 non-null  int64
 5   AGE                         30000 non-null  int64
 6   PAY_0                       30000 non-null  int64
 7   PAY_2                       30000 non-null  int64
 8   PAY_3                       30000 non-null  int64
 9   PAY_4                       30000 non-null  int64
 10  PAY_5                       30000 non-null  int64
 11  PAY_6                       30000 non-null  int64
 12  BILL_AMT1                   30000 non-null  int64
 13  BILL_AMT2                   30000 non-null  int64
 14  BILL_AMT3                   30000 non-null  int64
 15  BILL_AMT4                   30000 non-null  int64
 16  BILL_AMT5                   30000 non-null  int64
 17  BILL_AMT6                   30000 non-null  int64
 18  PAY_AMT1                    30000 non-null  int64
 19  PAY_AMT2                    30000 non-null  int64
 20  PAY_AMT3                    30000 non-null  int64
 21  PAY_AMT4                    30000 non-null  int64
 22  PAY_AMT5                    30000 non-null  int64
 23  PAY_AMT6                    30000 non-null  int64
 24  default payment next month  30000 non-null  int64
dtypes: int64(25)
memory usage: 5.7 MB
```

Fig. 6. Changing the data type into int.

2.3.2 Insight

Based on Fig.5 there is a huge imbalance in the target column which we'll have to be cautious about during splitting the dataset, instead of using *train_test_splot()*, we need to use *StratifiedShuffleSplit()* to ensure both classes are presented in the both train and test set.

3. Implementing Classification and Model Evaluation

The dataset will undergo *StratifiedShuffleSplit()* on Fig.7. to the most balanced class in each data set and the result splitting test is Training data set 80% and Test data set 20%.

```
Training Matrix Dimensionality: (24000, 24)
Testing Matrix Dimensionality: (6000, 24)
```

```
y_train.value_counts(normalize=True)
```

```
default payment next month
0    0.778792
1    0.221208
Name: proportion, dtype: float64
```

```
y_test.value_counts(normalize=True)
```

```
default payment next month
0    0.778833
1    0.221167
Name: proportion, dtype: float64
```

Fig.7. Data Splitting

After splitting and scaling both the train and test data, we will run the decided classification model and evaluate it. We'll go through the model one by one:

- **Logistic Regression**

  This model will estimate the probability of an example belonging to a specific class using a logistic function.Find Fig.8. for its evaluation matrix.

```
Logistic Regression Evaluation:
              precision    recall  f1-score   support

         0.0       0.82      0.97      0.89      4673
         1.0       0.69      0.24      0.36      1327

    accuracy                           0.81      6000
   macro avg       0.75      0.61      0.62      6000
weighted avg       0.79      0.81      0.77      6000
```

Fig.8. Logistic Regression Evaluation Matrix

- **KNN**

  This model classifies data points based on the majority class or average of its k closest training examples. Find Fig.9. for its evaluation matrix.

```
KNN Evaluation:
              precision    recall  f1-score   support

         0.0       0.83      0.89      0.86      4673
         1.0       0.48      0.36      0.41      1327

    accuracy                           0.77      6000
   macro avg       0.66      0.62      0.63      6000
weighted avg       0.75      0.77      0.76      6000
```

Fig.9. KNN Evaluation Matrix

- **Random Forest**

  Random forest is an ensemble method that aggregates the predictions of multiple decision trees. Find Fig.10. for its evaluation matrix.

```
Random Forest Evaluation:
              precision    recall  f1-score   support

         0.0       0.84      0.95      0.89      4673
         1.0       0.66      0.36      0.47      1327

    accuracy                           0.82      6000
   macro avg       0.75      0.66      0.68      6000
weighted avg       0.80      0.82      0.80      6000
```

Fig.10. Random Forest Evaluation Matrix

- **Gradient Boosting**

  This method will sequentially train a weak learners tree, where each corrects the errors of its predecessor. Find Fig.11. for its evaluation matrix.

```
Gradient Boosting Evaluation:
              precision    recall  f1-score   support

         0.0       0.84      0.95      0.89      4673
         1.0       0.67      0.36      0.47      1327

    accuracy                           0.82      6000
   macro avg       0.75      0.66      0.68      6000
weighted avg       0.80      0.82      0.80      6000
```

Fig.11. Gradient Boosting Evaluation Matrix

- **Stacking Ensemble**

  This model trains a meta-model by improving and aggregating the prediction of several base-models. Find Fig.12. for its evaluation matrix.

```
Stacking Ensemble Evaluation:
              precision    recall  f1-score   support

         0.0       0.84      0.95      0.89      4673
         1.0       0.68      0.36      0.47      1327

    accuracy                           0.82      6000
   macro avg       0.76      0.66      0.68      6000
weighted avg       0.80      0.82      0.80      6000
```

Fig.12. Stacking Ensemble Evaluation Matrix

4. Classification Model Recommendation

In order to summarize the model evaluation, we'll need to combine all the evaluation matrixs and create a heatmap for better data visualization.

---------- Final Metrics Comparison Matrix ----------

	Precision	Recall	Accuracy	F1-score	AUC
KNN	0.48	0.36	0.77	0.41	0.62
Gradient Boosting	0.67	0.36	0.82	0.47	0.66
Random Forest	0.66	0.36	0.82	0.47	0.66
Stacking Ensemble	0.68	0.36	0.82	0.47	0.66
Logistic Regression	0.69	0.24	0.81	0.36	0.61

Fig.13. Final Metrics Comparison Matrix

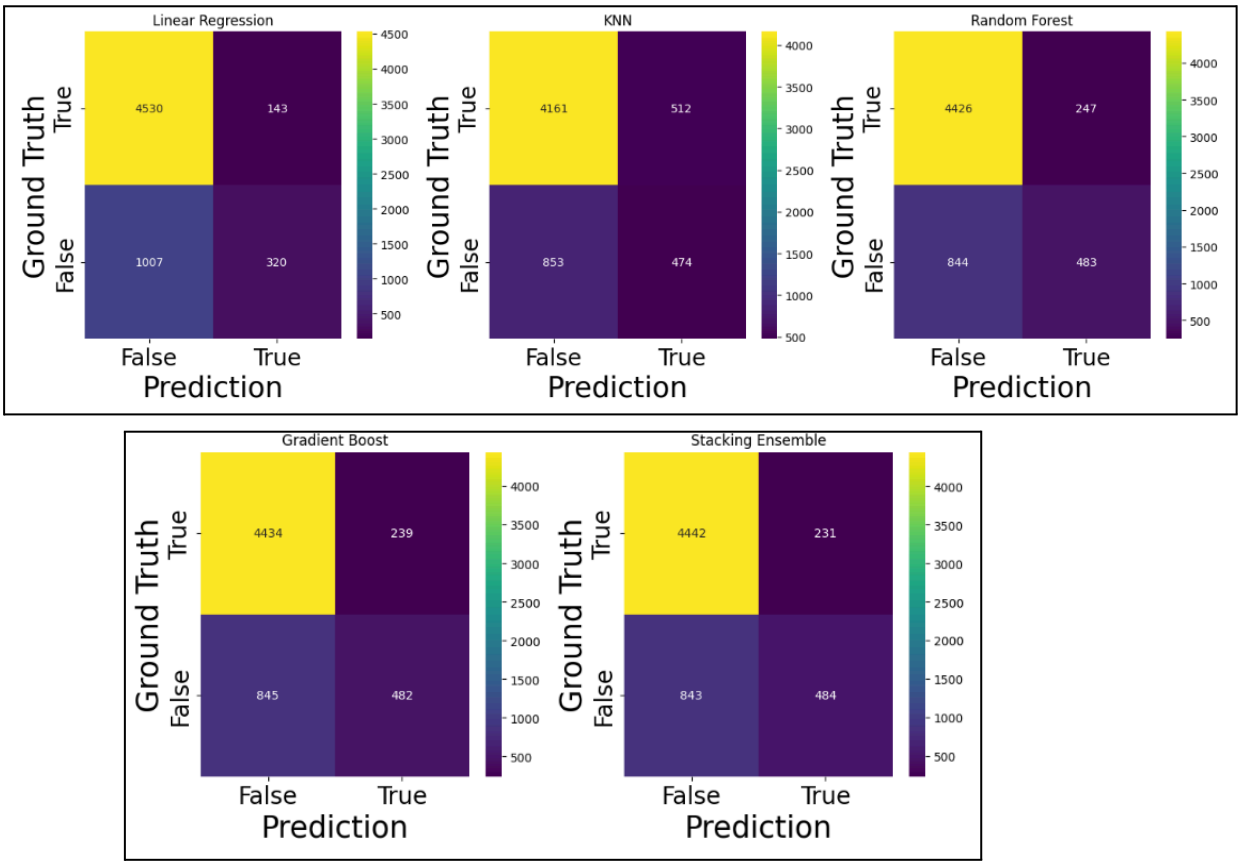

Fig.14. Heatmap from comparison matrix

Based on the graph and table, **Stacking Ensemble** indicates to be the most optimal model to predict for the CC dataset. Even though Stacking Ensemble, Random Forest, and Gradient Boosting have identical Recall values of 0.36. However, sStacking Ensemble has highest Precision (0.68) and F1-Score aggregate in 0.47 as the optimal equilibrium to detect failure payment and minimise the failure detention of potential safe customers.

5. Insight and Key Findings: Dataset Objective

We've already developed and executed 5 models on the data set. We found that the **Stacking Ensemble** is the most suitable to predict the binary categorical output. **However, Stacking Ensemble is highly susceptible to overfitting.** If the Level-0 base estimators (e.g., Random Forest, KNN) memorize the training matrix and output artificially perfect predictions, the Level-1 meta-model will learn to blindly trust these flawed probabilities, resulting in total predictive collapse on unseen testing data.

6. Model Plan of Improvement

In order to further improve the model, we can use **Out-of-Fold (OOF).** So, the meta-model must never be trained on the exact same predictions the base models generated from their training data.